

Algoritmos golosos - Ejercicios

Contenidos

Contenidos	1
Ejercicio 1. Planificación de tareas con <i>deadlines</i>	2
Descripción del problema	2
Restricciones	2
Objetivo	2
Especificación formal	2
Ejemplo	3
Solución mediante algoritmo Greedy	3
Pseudocódigo	3
Ejemplo de ejecución	4
Demostración de correctitud usando la técnica <i>Greedy Stays Ahead</i>	4
Ejercicio 2 (Viaje a mar del plata)	7
Algoritmo Greedy propuesto	7
Ejemplo de ejecución	8
Demostración de correctitud	8
Ejercicio 3: Minimización del producto escalar	11
Enunciado del problema	11
Algoritmo Greedy propuesto	11
Ejemplo	12
Demostración de correctitud	12

$$\sum_{i \in QC} \max(\max_{\text{workqueue}})$$

Ejercicio 1. Planificación de tareas con deadlines

Descripción del problema

Se tiene un conjunto de n tareas $\mathcal{T} = \{t_1, t_2, \dots, t_n\}$, donde cada tarea t_i tiene asociado un *deadline* $d_i \in \mathbb{Z}^+$ que representa el tiempo límite antes del cual debe completarse la tarea.

Restricciones

- Cada tarea requiere exactamente **una unidad de tiempo** para completarse.
- Solo se puede ejecutar **una tarea a la vez** (no existe paralelismo).
- El tiempo comienza en $t = 0$ y avanza en unidades discretas: $t \in \{0, 1, 2, \dots\}$.
- Una tarea t_i se considera completada exitosamente si y solo si termina su ejecución en un tiempo $f_i \leq d_i$, donde f_i es el tiempo de finalización de la tarea.
- Una vez iniciada una tarea, debe completarse sin interrupciones (no hay *preemption*).
- Las tareas pueden ejecutarse en cualquier orden.

Objetivo

Determinar una planificación (orden de ejecución) que **maximice el número total de tareas** que pueden completarse respetando sus respectivos *deadlines*.

Especificación formal

Entrada

- Un entero $n \in \mathbb{Z}^+$ representando el número de tareas.
- Un conjunto de *deadlines* $D = \{d_1, d_2, \dots, d_n\}$ donde $d_i \in \mathbb{Z}^+$ para todo $i \in \{1, 2, \dots, n\}$.

Salida

- El número máximo k de tareas que pueden completarse dentro de sus *deadlines*.
- (Opcional) Una secuencia $S = \{s_1, s_2, \dots, s_k\}$ que representa un orden válido de ejecución, donde $s_j \in \{1, 2, \dots, n\}$ indica el índice de la j -ésima tarea a ejecutar.

Ejemplo

Considere $n = 4$ tareas con *deadlines* $D = \{2, 1, 3, 2\}$.

Tarea	Deadline
t_1	2
t_2	1
t_3	3
t_4	2

Una solución óptima sería:

- Ejecutar t_2 en el intervalo $[0, 1)$ (se completa en tiempo 1, cumple con $d_2 = 1$)
- Ejecutar t_1 en el intervalo $[1, 2)$ (se completa en tiempo 2, cumple con $d_1 = 2$)
- Ejecutar t_3 en el intervalo $[2, 3)$ (se completa en tiempo 3, cumple con $d_3 = 3$)

Esta planificación completa 3 tareas exitosamente. La tarea t_4 no puede ejecutarse sin violar su *deadline*.

Solución mediante algoritmo Greedy

Para resolver el problema de planificación de tareas con *deadlines*, utilizaremos un algoritmo greedy con la siguiente estrategia:

1. Ordenar todas las tareas por sus *deadlines* en orden creciente.
2. Recorrer las tareas en este orden y ejecutar cada tarea si es posible completarla antes de su *deadline*.

Pseudocódigo

Algoritmo: MaxTareas(T, D)

Entrada: T = conjunto de n tareas

D = vector de *deadlines* $d[1..n]$

Salida: Número máximo de tareas que se pueden completar

1. Ordenar las tareas por *deadline*: $d[i_1] \leq d[i_2] \leq \dots \leq d[i_n]$
2. $S \leftarrow \{\}$ // Conjunto de tareas seleccionadas
3. $t_{\text{actual}} \leftarrow 0$ // Tiempo actual
- 4.
5. Para cada tarea t_j en orden de *deadline* creciente hacer:
6. Si $t_{\text{actual}} + 1 \leq d[j]$ entonces:
7. $S \leftarrow S \cup \{t_j\}$
8. $t_{\text{actual}} \leftarrow t_{\text{actual}} + 1$

9. Fin Si
10. Fin Para
- 11.
12. Retornar $|S|$

Complejidad temporal: $O(n \log n)$ debido al ordenamiento inicial.

Ejemplo de ejecución

Consideremos el ejemplo con 4 tareas y *deadlines* $D = \{2, 1, 3, 2\}$:

Iteración	Tarea	Deadline	Tiempo actual
Inicial	-	-	0
1	t_2	1	$0 \rightarrow 1$
2	t_1	2	$1 \rightarrow 2$
3	t_4	2	No factible ($2 + 1 > 2$)
4	t_3	3	$2 \rightarrow 3$

Solución: $\{t_2, t_1, t_3\}$ con 3 tareas completadas.

Demostración de correctitud usando la técnica Greedy Stays Ahead

Para demostrar que nuestro algoritmo greedy produce una solución óptima, utilizaremos la técnica conocida como “*Greedy Stays Ahead*” o “*El Greedy se mantiene adelante*”. Esta técnica consiste en demostrar que en cada paso, la solución greedy está al menos tan bien posicionada como cualquier solución óptima.

Notación y preliminares

- Sea $\mathcal{G} = \{g_1, g_2, \dots, g_m\}$ la solución obtenida por nuestro algoritmo greedy, ordenada por tiempo de ejecución (y por tanto, por *deadline*).
- Sea $\mathcal{O} = \{o_1, o_2, \dots, o_n\}$ una solución óptima arbitraria.
- Denotamos $d(t_i)$ como el *deadline* de la tarea t_i .

Observación clave: Podemos reordenar cualquier solución factible sin perder factibilidad mediante el siguiente argumento de intercambio:

Lema (Intercambio sin pérdida): *Si en una solución factible ejecutamos la tarea t_i antes que t_j , donde $d(t_i) > d(t_j)$, podemos intercambiar su orden de ejecución manteniendo la factibilidad.*

Proof. Si t_i se ejecuta en tiempo s y t_j en tiempo $s' > s$, entonces:

- t_i cumple: $s + 1 \leq d(t_i)$

- t_j cumple: $s' + 1 \leq d(t_j)$

Después del intercambio:

- t_j en tiempo s : $s + 1 \leq s' + 1 \leq d(t_j)$
- t_i en tiempo s' : $s' + 1 \leq d(t_j) < d(t_i)$

■

Por lo tanto, podemos asumir sin pérdida de generalidad que $\mathcal{O}$ está ordenada por *deadlines*: $d(o_1) \leq d(o_2) \leq \dots \leq d(o_n)$.

Lema principal: Greedy Stays Ahead

Lema (*Greedy Stays Ahead*): Para todo $i \in \{1, 2, \dots, \min(m, n)\}$, se cumple que $d(g_i) \leq d(o_i)$.

Proof. Procederemos por inducción sobre i .

Caso base ($i = 1$): La tarea g_1 es la tarea con el menor *deadline* que puede ejecutarse comenzando en tiempo 0. Por definición del algoritmo greedy, $d(g_1)$ es mínimo entre todas las tareas factibles. Como o_1 también es una tarea ejecutada en una solución factible, necesariamente $d(g_1) \leq d(o_1)$.

Paso inductivo: Supongamos que la propiedad se cumple hasta $k - 1$, es decir, $d(g_j) \leq d(o_j)$ para todo $j < k$. Queremos demostrar que $d(g_k) \leq d(o_k)$.

Por hipótesis inductiva, el algoritmo greedy ha ejecutado las primeras $k - 1$ tareas con *deadlines* menores o iguales que las primeras $k - 1$ tareas de la solución óptima. En el paso k :

1. El tiempo de finalización de g_{k-1} es $k - 1 \leq$ tiempo de finalización de o_{k-1} .
2. La tarea o_k es factible para la solución óptima después de ejecutar $\{o_1, \dots, o_{k-1}\}$.
3. Como $d(o_{k-1}) \leq d(o_k)$ y el greedy elige la tarea con menor *deadline* entre las factibles, si o_k fuera candidata para el greedy, este la habría considerado.
4. El greedy eligió g_k con el menor *deadline* posible entre todas las tareas factibles en ese momento.
5. Por lo tanto, $d(g_k) \leq d(o_k)$.

■

Teorema de optimalidad

Teorema 1: *El algoritmo greedy produce una solución óptima para el problema de planificación de tareas con deadlines.*

Proof. Sea $\mathcal{G}$ la solución greedy con m tareas y $\mathcal{O}$ una solución óptima con n tareas. Debemos demostrar que $m = n$.

Supongamos por contradicción que $m < n$.

Por el Lema *Greedy Stays Ahead*, sabemos que:

- Para todo $i \leq m$: $d(g_i) \leq d(o_i)$
- El tiempo de finalización de g_m es m
- El tiempo de finalización de o_m es al menos m (ya que ejecuta m tareas)

Consideremos la tarea o_{m+1} :

1. Por el lema, $d(g_m) \leq d(o_m) \leq d(o_{m+1})$ (por el ordenamiento)
2. La tarea o_{m+1} se ejecuta en tiempo $m+1$ en la solución óptima, por lo que $m+1 \leq d(o_{m+1})$
3. Esto significa que después de ejecutar $\mathcal{G}$, en tiempo m , la tarea o_{m+1} (o cualquier otra con el mismo *deadline*) sería factible
4. El algoritmo greedy habría agregado esta tarea a su solución

Esto contradice el hecho de que el greedy se detuvo con m tareas. Por lo tanto, $m = n$ y el algoritmo greedy es óptimo. ■

Ejercicio 2 (Viaje a mar del plata)

Tomás quiere viajar de Buenos Aires a Mar del Plata en su flamante Renault 12. Como está preocupado por la autonomía de su vehículo, se tomó el tiempo de anotar las distintas estaciones de servicio que se encuentran en el camino. Modeló el mismo como un segmento de 0 a M , donde Buenos aires está en el kilómetro 0, Mar del Plata en el M , y las distintas estaciones de servicio están ubicadas en los kilómetros $0 = x_1 \leq x_2 \leq \dots x_n \leq M$.

Razonablemente, Tomás quiere minimizar la cantidad de paradas para cargar nafta. Él sabe que su auto es capaz de hacer hasta C kilómetros con el tanque lleno, y que al comenzar el viaje este está vacío.

- Proponer un algoritmo *greedy* que indique cuál es la cantidad mínima de paradas para cargar nafta que debe hacer Tomás, y que aparte devuelva el conjunto de estaciones en las que hay que detenerse. Probar su correctitud.
- Dar una implementación de complejidad temporal $O(n)$ del algoritmo.

Algoritmo Greedy propuesto

La estrategia greedy es simple pero efectiva: **siempre avanzar hasta la estación más lejana posible dentro del alcance C kilómetros.**

Descripción intuitiva

Comenzando desde Buenos Aires (o desde la última estación donde cargamos combustible), miramos todas las estaciones que están dentro de nuestro alcance de C kilómetros y elegimos la más lejana. Repetimos este proceso hasta llegar a Mar del Plata.

Pseudocódigo

```

Algoritmo: MinParadas( $x[]$ ,  $n$ ,  $C$ ,  $M$ )
Entrada:  $x[1..n]$  = posiciones de estaciones ( $x[0] = 0$  implícito)
 $C$  = alcance máximo con tanque lleno
 $M$  = distancia total a Mar del Plata
Salida: Conjunto de estaciones donde parar

1.  posicion_actual <- 0
2.  estaciones <- { $x[1]$ } // Empezamos cargando en Buenos Aires
3.  idx <- 1
4.
5.  Mientras posicion_actual +  $C$  <  $M$  hacer:
6.      ultima_alcanzable <- idx - 1
7.
8.      // Buscar la estación más lejana alcanzable
9.      Mientras idx <=  $n$  Y  $x[idx] \leq$  posicion_actual +  $C$  hacer:

```

```

10.         ultima_alcanzable <- idx
11.         idx <- idx + 1
12.     Fin Mientras
13.
14.     Si ultima_alcanzable == idx - 1 entonces:
15.         // No hay estación alcanzable
16.         Retornar "No hay solución"
17.     Fin Si
18.
19.     estaciones <- estaciones U {x[ultima_alcanzable]}
20.     posicion_actual <- x[ultima_alcanzable]
21. Fin Mientras
22.
23. Retornar estaciones

```

Complejidad: $O(n)$, ya que cada estación se examina a lo sumo una vez.

Ejemplo de ejecución

Consideremos un viaje con los siguientes datos:

- Distancia total $M = 400$ km
- Alcance $C = 150$ km
- Estaciones en: $\{0, 80, 140, 200, 280, 350, 400\}$

Iteración	Posición actual	Alcance hasta	Estación elegida
1	0	150	140 (más lejana ≤ 150)
2	140	290	280 (más lejana ≤ 290)
3	280	430	400 (destino alcanzable)

Solución: Paradas en las estaciones ubicadas en km 0, 140, y 280. Total: 3 paradas.

Demostración de correctitud

Presentaremos dos demostraciones complementarias de la optimalidad del algoritmo.

Demostración 1: propiedad de elección Greedy y subestructura óptima

Lema 1 (Propiedad de elección Greedy). *Existe una solución óptima que incluye la primera elección del algoritmo greedy.*

Demostración. Sea g_1 la primera estación elegida por nuestro algoritmo greedy (la estación más lejana alcanzable desde el origen con alcance C).

Sea $\mathcal{O} = \{o_1, o_2, \dots, o_k\}$ una solución óptima cualquiera, donde las estaciones están ordenadas por posición.

Caso 1: Si $o_1 = g_1$, no hay nada que probar.

Caso 2: Si $o_1 \neq g_1$, entonces necesariamente $x_{o_1} < x_{g_1}$ (ya que g_1 es la más lejana alcanzable).

Construimos una nueva solución $\mathcal{O}' = \{g_1\} \cup \{o_i \in \mathcal{O} : x_{o_i} > x_{g_1}\}$.

Veamos que $\mathcal{O}'$ es factible:

- g_1 es alcanzable desde el origen por definición
- Para cada estación o_i con $x_{o_i} > x_{g_1}$: Si era alcanzable desde o_1 en la solución original, también lo será desde g_1 , pues:

$$x_{o_i} - x_{g_1} < x_{o_i} - x_{o_1} \leq C$$

Como $|\mathcal{O}'| \leq |\mathcal{O}|$ y $\mathcal{O}$ es óptima, entonces $|\mathcal{O}'| = |\mathcal{O}|$, por lo que $\mathcal{O}'$ también es óptima y contiene a g_1 . $\square$

Lema 2 (Subestructura óptima). Si $\mathcal{O}$ es una solución óptima para llegar de 0 a M , y o_k es una estación en $\mathcal{O}$, entonces $\mathcal{O} \setminus \{o_1, \dots, o_k\}$ es una solución óptima para el subproblema de ir de x_{o_k} a M .

Demostración. Por contradicción. Supongamos que existe una solución $\mathcal{S}$ para ir de x_{o_k} a M con menos paradas que $\mathcal{O} \setminus \{o_1, \dots, o_k\}$.

Entonces $\{o_1, \dots, o_k\} \cup \mathcal{S}$ sería una solución factible para el problema original con menos paradas que $\mathcal{O}$, contradiciendo la optimalidad de $\mathcal{O}$. $\square$

Teorema. El algoritmo greedy produce una solución óptima.

Demostración (Construcción inductiva).

Sea $\mathcal{G} = \{g_1, g_2, \dots, g_k\}$ la solución producida por el algoritmo greedy, donde g_i denota la i -ésima estación elegida.

Probaremos por inducción que para cada $i \in \{1, \dots, k\}$, existe una solución óptima que contiene las primeras i elecciones greedy $\{g_1, \dots, g_i\}$.

Caso base ($i = 1$): Por el Lema 1, existe una solución óptima $\mathcal{O}_1$ que contiene g_1 (la primera elección greedy).

Hipótesis inductiva: Supongamos que existe una solución óptima $\mathcal{O}_i$ que contiene $\{g_1, g_2, \dots, g_i\}$.

Paso inductivo: Necesitamos demostrar que existe una solución óptima que contiene $\{g_1, \dots, g_i, g_{i+1}\}$.

Por el Lema 2, si $\mathcal{O}_i$ es óptima para ir de 0 a M , entonces $\mathcal{O}_i \setminus \{g_1, \dots, g_i\}$ es una solución óptima para el subproblema de ir de x_{g_i} a M .

Ahora, aplicamos el mismo argumento del Lema 1 al subproblema:

- g_{i+1} es la estación más lejana alcanzable desde x_{g_i}
- Si $\mathcal{O}_i \setminus \{g_1, \dots, g_i\}$ no comienza con g_{i+1} , podemos reemplazar su primera estación por g_{i+1} sin aumentar el número de paradas (mismo argumento que en el Lema 1)

Por lo tanto, existe una solución óptima $\mathcal{O}_{i+1} = \{g_1, \dots, g_i, g_{i+1}\} \cup \mathcal{S}$ donde $\mathcal{S}$ es una solución óptima para ir de $x_{g_{i+1}}$ a M .

Conclusión: Por inducción, existe una solución óptima que contiene todas las elecciones greedy $\{g_1, \dots, g_k\}$. Como el algoritmo greedy termina cuando alcanza M , y ninguna solución puede usar menos paradas que una óptima, concluimos que $\mathcal{G}$ es una solución óptima. $\square$

Demostración 2: técnica Greedy Stays Ahead

Esta técnica demuestra que el algoritmo greedy siempre “avanza más” que cualquier solución óptima en cada paso.

Lema 3 (Greedy Stays Ahead). Sean $\mathcal{G} = \{g_1, \dots, g_m\}$ la solución greedy y $\mathcal{O} = \{o_1, \dots, o_n\}$ una solución óptima. Para todo $i \in \{1, \dots, \min(m, n)\}$, se cumple que $x_{g_i} \geq x_{o_i}$.

Demostración. Procederemos por inducción sobre i .

Caso base ($i = 1$): Por definición del algoritmo greedy, g_1 es la estación más lejana alcanzable desde el origen. Como o_1 también debe ser alcanzable desde el origen (está en una solución factible), necesariamente $x_{g_1} \geq x_{o_1}$.

Paso inductivo: Asumimos que $x_{g_k} \geq x_{o_k}$ para todo $k \leq i$. Queremos probar que $x_{g_{i+1}} \geq x_{o_{i+1}}$. Sabemos que:

- o_{i+1} es alcanzable desde o_i : $x_{o_{i+1}} - x_{o_i} \leq C$
- Por hipótesis inductiva: $x_{g_i} \geq x_{o_i}$
- Por lo tanto: $x_{o_{i+1}} - x_{g_i} \leq x_{o_{i+1}} - x_{o_i} \leq C$

Esto significa que o_{i+1} está dentro del alcance desde g_i . Como el algoritmo greedy elige la estación más lejana posible desde g_i , tenemos que $x_{g_{i+1}} \geq x_{o_{i+1}}$. $\square$

Teorema (Optimalidad). El algoritmo greedy usa el mínimo número de paradas.

Demostración. Sea $\mathcal{G}$ la solución greedy con m paradas y $\mathcal{O}$ una solución óptima con n paradas.

Supongamos por contradicción que $m > n$ (el greedy usa más paradas).

Por el Lema 3, sabemos que $x_{g_n} \geq x_{o_n}$.

Como la solución óptima llega a M desde o_n , tenemos que $M - x_{o_n} \leq C$.

Por lo tanto: $M - x_{g_n} \leq M - x_{o_n} \leq C$

Esto significa que desde g_n se puede llegar directamente a M , por lo que el algoritmo greedy no necesitaría hacer más paradas después de g_n . Esto contradice que $m > n$.

Por lo tanto, $m \leq n$. Como n es el número óptimo de paradas, concluimos que $m = n$. $\square$

Ejercicio 3: Minimización del producto escalar

Enunciado del problema

Dados dos vectores $v, w \in \mathbb{R}^n$, queremos encontrar permutaciones σ y τ de sus coordenadas que minimicen el producto escalar entre los vectores reordenados.

Entrada:

- Dos vectores $v = (v_1, v_2, \dots, v_n)$ y $w = (w_1, w_2, \dots, w_n)$ en $\mathbb{R}^n$

Salida:

- Permutaciones $\sigma, \tau : \{1, \dots, n\} \rightarrow \{1, \dots, n\}$ tales que el producto escalar

$$\langle v_\sigma, w_\tau \rangle = \sum_{i=1}^n v_{\sigma(i)} \cdot w_{\tau(i)}$$

sea mínimo, donde $v_\sigma = (v_{\sigma(1)}, \dots, v_{\sigma(n)})$ y $w_\tau = (w_{\tau(1)}, \dots, w_{\tau(n)})$

Objetivo: Diseñar un algoritmo eficiente que encuentre las permutaciones óptimas y demostrar su correctitud.

Algoritmo Greedy propuesto

Estrategia: Ordenar un vector de forma creciente y el otro de forma decreciente, luego emparejar las coordenadas en ese orden.

Algoritmo

Algoritmo: MinimizarProductoEscalar(v, w)

Entrada: Vectores v, w en $\mathbb{R}^n$

Salida: Valor mínimo del producto escalar y las permutaciones σ, τ

```

1. índices_v <- ObtenerÍndicesOrdenados(v, creciente)
2. índices_w <- ObtenerÍndicesOrdenados(w, decreciente)
3. v_ordenado <- Ordenar v de menor a mayor
4. w_ordenado <- Ordenar w de mayor a menor
5. producto_min <- 0
6. Para i = 1 hasta n:
7.     producto_min <- producto_min + v_ordenado[i] * w_ordenado[i]
8. sigma <- índices_v      // permutación que ordena v
9. tau <- índices_w        // permutación que ordena w
10. Retornar (producto_min, sigma, tau)
```

Complejidad: $O(n \log n)$ debido a los ordenamientos.

Ejemplo

Sean $v = (3, 1, 4, 2)$ y $w = (5, 2, 8, 1)$.

Paso 1: Ordenar

- v ordenado creciente: $(1, 2, 3, 4)$
- w ordenado decreciente: $(8, 5, 2, 1)$

Paso 2: Calcular producto escalar mínimo

$$\langle v_{\text{ord}}, w_{\text{ord}} \rangle = 1 \cdot 8 + 2 \cdot 5 + 3 \cdot 2 + 4 \cdot 1 = 8 + 10 + 6 + 4 = 28$$

Verificación: Con el orden original:

$$\langle v, w \rangle = 3 \cdot 5 + 1 \cdot 2 + 4 \cdot 8 + 2 \cdot 1 = 15 + 2 + 32 + 2 = 51$$

El algoritmo greedy produce efectivamente un valor menor (28 vs 51).

Demostración de correctitud

Teorema principal

Teorema. Sean $v = (v_1, \dots, v_n)$ y $w = (w_1, \dots, w_n)$ dos vectores en $\mathbb{R}^n$. El producto escalar se minimiza cuando ordenamos las coordenadas de v en orden creciente y las de w en orden decreciente (o viceversa).

Lema de reordenamiento

Lema (Desigualdad de reordenamiento). Sean $a_1 \leq a_2 \leq \dots \leq a_n$ y $b_1 \leq b_2 \leq \dots \leq b_n$ dos secuencias ordenadas. Para cualquier permutación σ de $\{1, \dots, n\}$, se cumple:

$$\sum_{i=1}^n a_i \cdot b_{n+1-i} \leq \sum_{i=1}^n a_i \cdot b_{\sigma(i)} \leq \sum_{i=1}^n a_i \cdot b_i$$

Es decir, el producto escalar se minimiza emparejando el menor con el mayor, y se maximiza emparejando elementos en el mismo orden.

Demostración por argumento de intercambio

Demostración del teorema.

Sin pérdida de generalidad, supongamos que hemos ordenado:

- $v_1 \leq v_2 \leq \dots \leq v_n$
- $w_1 \geq w_2 \geq \dots \geq w_n$

Queremos demostrar que esta configuración minimiza el producto escalar. Procederemos por contradicción usando un argumento de intercambio.

Supongamos que existe otra permutación $\mathcal{O}$ de las coordenadas que produce un producto escalar menor. En esta permutación óptima $\mathcal{O}$, las coordenadas están emparejadas como:

$$\hat{v}_1, \hat{v}_2, \dots, \hat{v}_n$$

$$\hat{w}_1, \hat{w}_2, \dots, \hat{w}_n$$

donde $\{\hat{v}_i\}$ y $\{\hat{w}_i\}$ son reordenamientos de $\{v_i\}$ y $\{w_i\}$ respectivamente.

Paso clave: Si en la solución $\mathcal{O}$ existen índices $i < j$ tales que:

- $\hat{v}_i > \hat{v}_j$ (las coordenadas de v no están en orden creciente), o
- $\hat{w}_i < \hat{w}_j$ (las coordenadas de w no están en orden decreciente)

Entonces podemos mejorar la solución intercambiando.

Análisis del intercambio: Supongamos que $\hat{v}_i > \hat{v}_j$ para algún $i < j$. Consideremos el efecto de intercambiar $\hat{v}_i$ y $\hat{v}_j$:

Producto antes del intercambio en las posiciones i, j :

$$S_{\text{antes}} = \hat{v}_i \cdot \hat{w}_i + \hat{v}_j \cdot \hat{w}_j$$

Producto después del intercambio:

$$S_{\text{después}} = \hat{v}_j \cdot \hat{w}_i + \hat{v}_i \cdot \hat{w}_j$$

La diferencia es:

$$\begin{aligned} S_{\text{después}} - S_{\text{antes}} &= \hat{v}_j \cdot \hat{w}_i + \hat{v}_i \cdot \hat{w}_j - \hat{v}_i \cdot \hat{w}_i - \hat{v}_j \cdot \hat{w}_j \\ &= (\hat{v}_j - \hat{v}_i)(\hat{w}_i - \hat{w}_j) \end{aligned} \quad \begin{matrix} (1) \\ (2) \end{matrix}$$

Como $\hat{v}_i > \hat{v}_j$, tenemos $\hat{v}_j - \hat{v}_i < 0$.

Para que la solución $\mathcal{O}$ sea óptima, necesitamos que el intercambio no mejore la solución, es decir:

$$S_{\text{después}} - S_{\text{antes}} \geq 0$$

Esto requiere:

$$(\hat{v}_j - \hat{v}_i)(\hat{w}_i - \hat{w}_j) \geq 0$$

Dado que $\hat{v}_j - \hat{v}_i < 0$, necesitamos $\hat{w}_i - \hat{w}_j \leq 0$, es decir, $\hat{w}_i \leq \hat{w}_j$.

Conclusión: Si la solución es óptima y no se puede mejorar mediante intercambios, entonces:

- Si $i < j$ y $\hat{v}_i > \hat{v}_j$, entonces $\hat{w}_i \leq \hat{w}_j$
- Por un argumento simétrico, si $\hat{w}_i < \hat{w}_j$, entonces $\hat{v}_i \geq \hat{v}_j$

Aplicando este argumento repetidamente, concluimos que en cualquier solución óptima:

- Las coordenadas de v deben estar ordenadas de forma opuesta a las de w
- La configuración óptima es: menor de v con mayor de w , segundo menor de v con segundo mayor de w , etc.

Esto es exactamente lo que produce nuestro algoritmo greedy. $\square$

Observación sobre el caso del máximo. Aunque para este ejercicio solo necesitamos la minimización, el Lema de reordenamiento también establece que el máximo se obtiene emparejando elementos en el mismo orden (menor con menor, mayor con mayor). La demostración es análoga: al realizar el mismo argumento de intercambio, las desigualdades se invierten y se concluye que cualquier desviación del orden concordante incrementa el producto escalar.

Propiedad de subestructura óptima

Observación. El problema exhibe subestructura óptima: una vez que emparejamos el menor elemento de v con el mayor de w , el problema restante es minimizar el producto escalar de los vectores de dimensión $n - 1$ obtenidos al eliminar estos elementos.

Esta propiedad justifica la construcción inductiva de la solución y confirma que el algoritmo greedy produce una solución globalmente óptima.

Demostración por inducción de la optimalidad global

Teorema (Construcción inductiva). *El algoritmo greedy que empareja iterativamente el menor elemento restante de v con el mayor elemento restante de w produce una solución globalmente óptima.*

Demostración por inducción sobre n .

Caso base ($n = 1$): Para vectores de dimensión 1, solo existe una permutación posible, por lo que el algoritmo es trivialmente óptimo.

Caso base ($n = 2$): Sean $v = (v_1, v_2)$ y $w = (w_1, w_2)$ con $v_1 \leq v_2$ y $w_1 \geq w_2$. Existen dos posibles emparejamientos:

- Greedy: $v_1 \cdot w_1 + v_2 \cdot w_2$
- Alternativo: $v_1 \cdot w_2 + v_2 \cdot w_1$

La diferencia es:

$$(v_1 \cdot w_2 + v_2 \cdot w_1) - (v_1 \cdot w_1 + v_2 \cdot w_2) = (v_2 - v_1)(w_1 - w_2) \geq 0$$

Ya que $(v_2 - v_1) \geq 0$ y $(w_1 - w_2) \geq 0$. Por lo tanto, el emparejamiento greedy es óptimo.

Hipótesis inductiva: Supongamos que para vectores de dimensión $k < n$, el algoritmo greedy produce la solución óptima.

Paso inductivo: Sean $v, w \in \mathbb{R}^n$ con elementos ordenados $v_1 \leq \dots \leq v_n$ y $w_1 \geq \dots \geq w_n$.

Consideremos cualquier solución óptima $\mathcal{O}^*$. Por el argumento de intercambio demostrado anteriormente, sabemos que en cualquier solución óptima, v_1 debe estar emparejado con algún

w_j tal que no existe un $w_k > w_j$ disponible que mejore la solución. Dado que w_1 es el máximo, v_1 debe estar emparejado con w_1 en alguna solución óptima.

Una vez fijado el emparejamiento (v_1, w_1) , el problema restante es minimizar:

$$\sum_{i=2}^n v_{\sigma(i)} \cdot w_{\tau(i)}$$

Este es un problema de minimización de producto escalar en $\mathbb{R}^{n-1}$ con los vectores $(v_2, \dots, v_n)$ y $(w_2, \dots, w_n)$.

Por hipótesis inductiva, la solución óptima para este subproblema es emparejar:

- v_2 (menor restante) con w_2 (mayor restante)
- v_3 con w_3
- Y así sucesivamente...

Por lo tanto, la solución globalmente óptima es:

$$\min \sum_{i=1}^n v_{\sigma(i)} \cdot w_{\tau(i)} = \sum_{i=1}^n v_i \cdot w_i$$

donde v_i y w_i están ordenados de forma opuesta, que es exactamente lo que produce nuestro algoritmo greedy. $\square$